XLAB Steampunk

Steampunk Spotter

Ansible Beyond the YAML

Understanding Automation as a Supply Chain

Gregor Berginc

CEO

gregor.berginc@xlab.si

XLAB Steampunk

Steampunk Spotter

Ansible Automation as a Supply Chain

Governance as the enabler of speed at scale

Gregor Berginc

CEO

gregor.berginc@xlab.si

We breathe Ansible

Written gazillion Ansible Playbooks

Developed 11 Ansible Collections

1000s of playbooks scanned with Steampunk Spotter every month

Spotter used by Fortune 500 companies

When you **run** an Ansible playbook, you are **executing a software supply chain**

Why governance?

Automation has become **critical infrastructure**

Automation runs with **high privileges**

Automation changes systems **at scale**

Governance often **lags behind**

XLAB Steampunk

EXAMPLE

Common security risks in Ansible Playbooks

```
---
- hosts: localhost
  tasks:
    - name: execute any command, given by -e switch
      ansible.builtin.command: "{{ cmd }}"
```

Trust in unvalidated variables

A single variable from a **vars file** or unchecked input can turn a valid Ansible Playbook into a destructive one. Assumptions about “trusted users” or “safe defaults” are where things go wrong.

SECURITY RISK

Exposing sensitive data to full-scale system failure.

EXAMPLE

Common security risks in Ansible Infrastructure

```
---

- hosts: localhost
  become: true
  vars:
   working_file: /tmp/playbook.zip
   working_folder: /tmp/playbook_folder
  tasks:
   - servicenow.itsm.incident_info:
       instance: "{{ sn_instance }}"
       query:
         - state: = new
     register: incident_result

   - ansible.builtin.set_fact:
       ones_with_attachments:
"{{ incident_result.records |
selectattr('attachments') | list }}"

   - ansible.builtin.include_tasks: act.yml
     vars:
```

Running playbooks with unnecessary root privileges

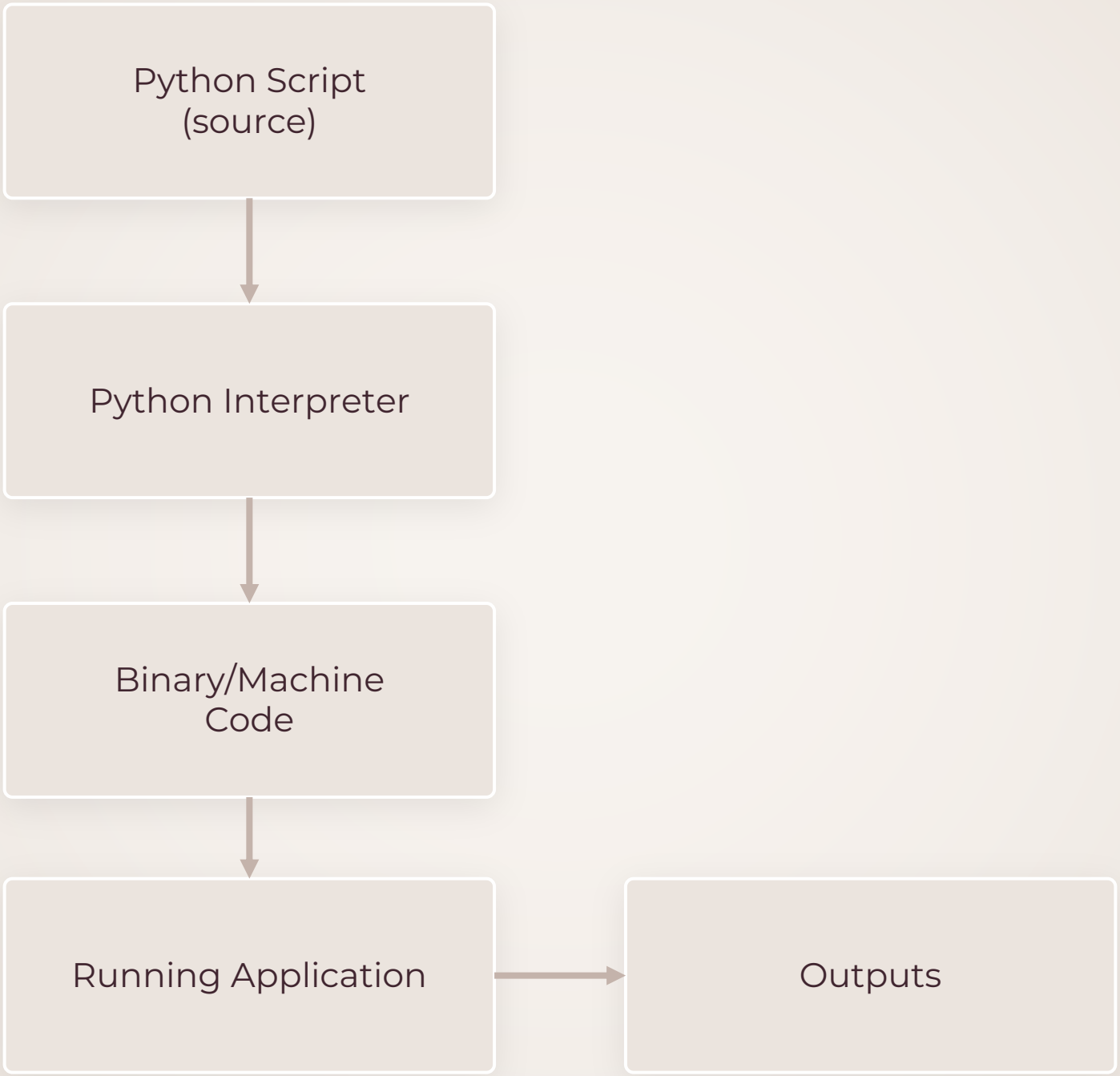
The playbook is executed entirely with administrator privileges.

SECURITY RISK

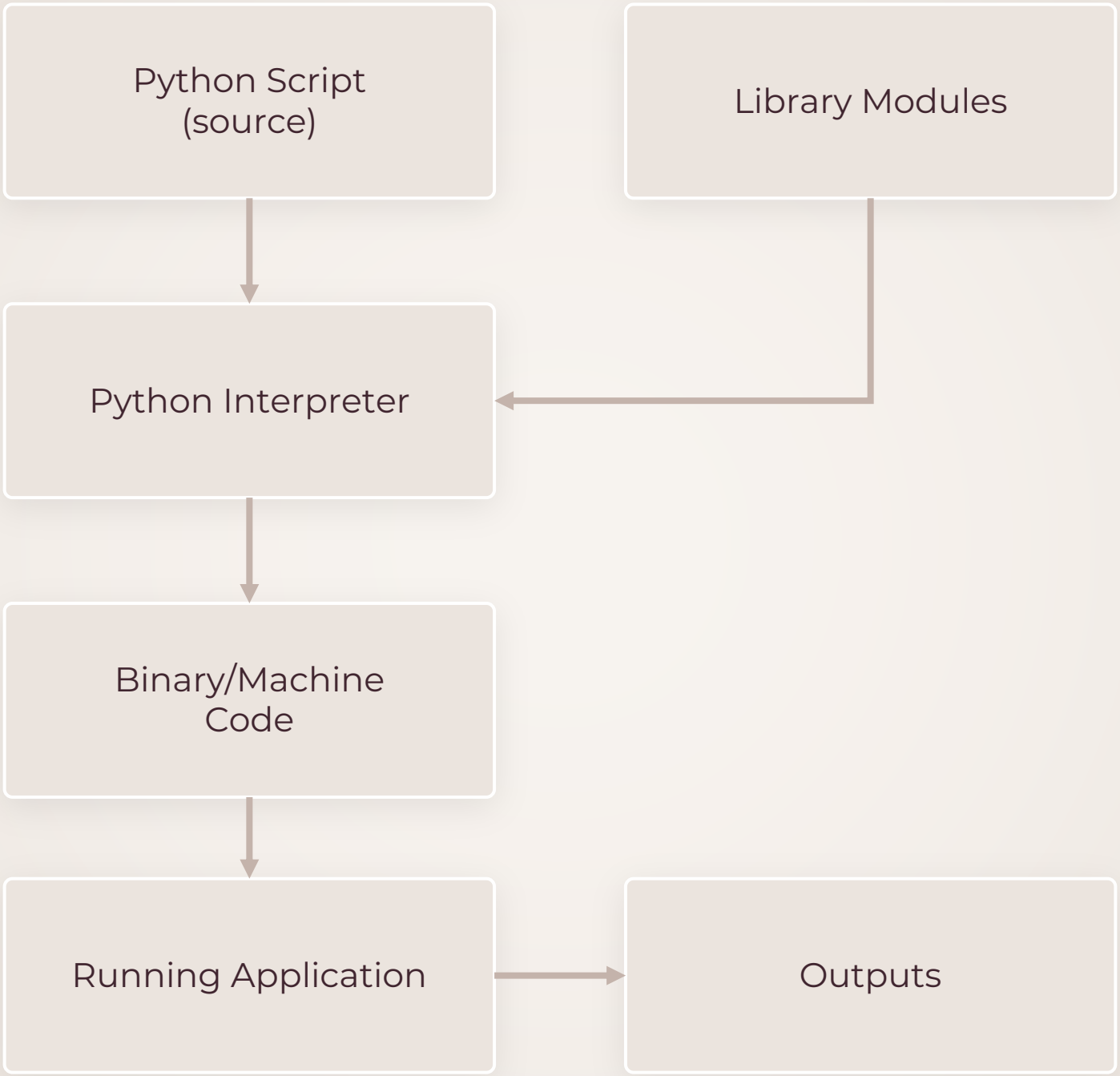
A task or dependency could unintentionally cause system issues, especially if a future change adds a new task without realizing the playbook runs as admin.

XLAB Steampunk

Python Example

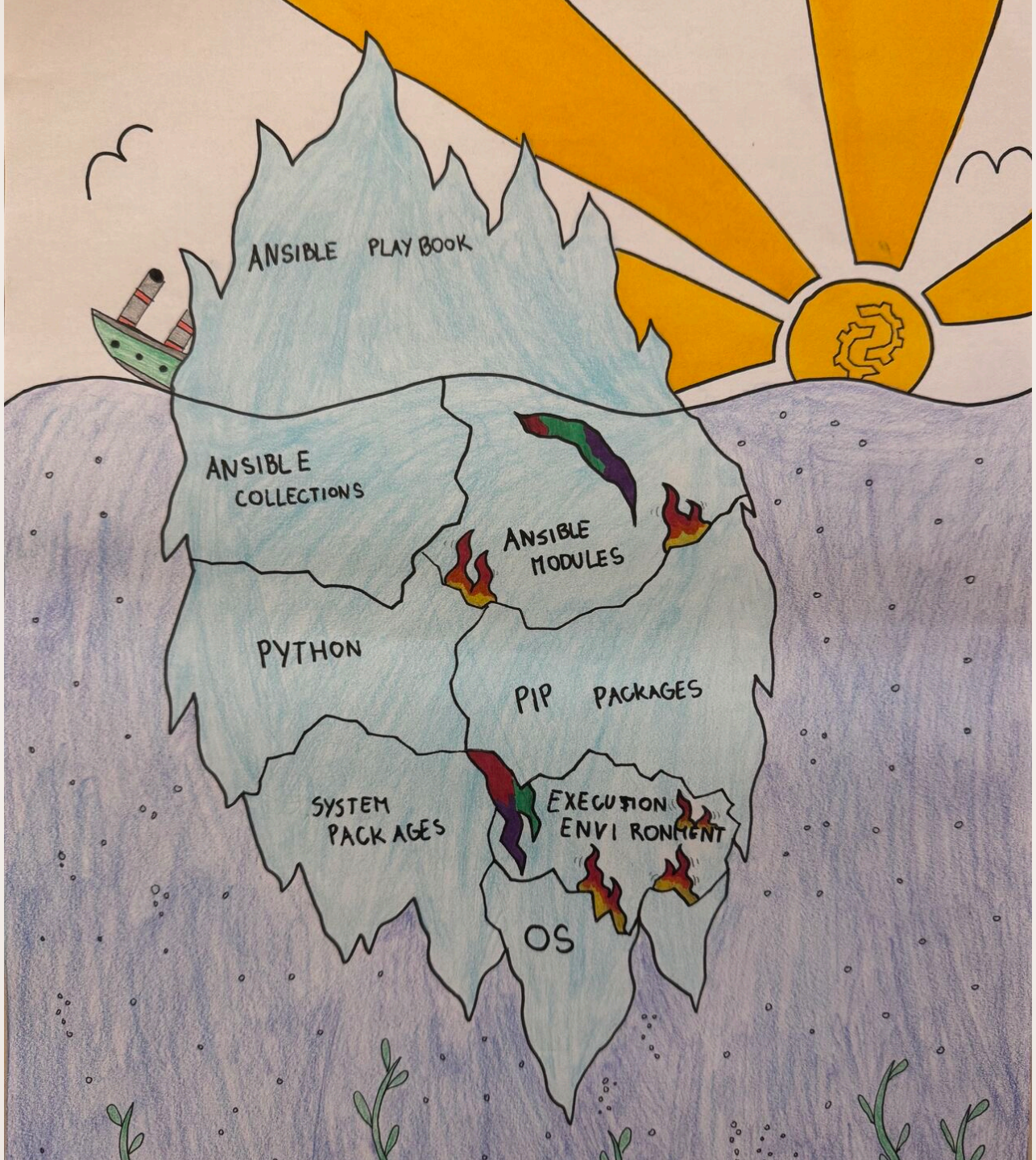


XLAB
Steampunk



XLAB
Steampunk

What actually runs when you execute an Ansible playbook?


ANSIBLE PLAYBOOK
ANSIBLE COLLECTIONS
ANSIBLE MODULES
PYTHON
PIP PACKAGES
SYSTEM PACKAGES
EXECUTION ENVIRONMENT
OS

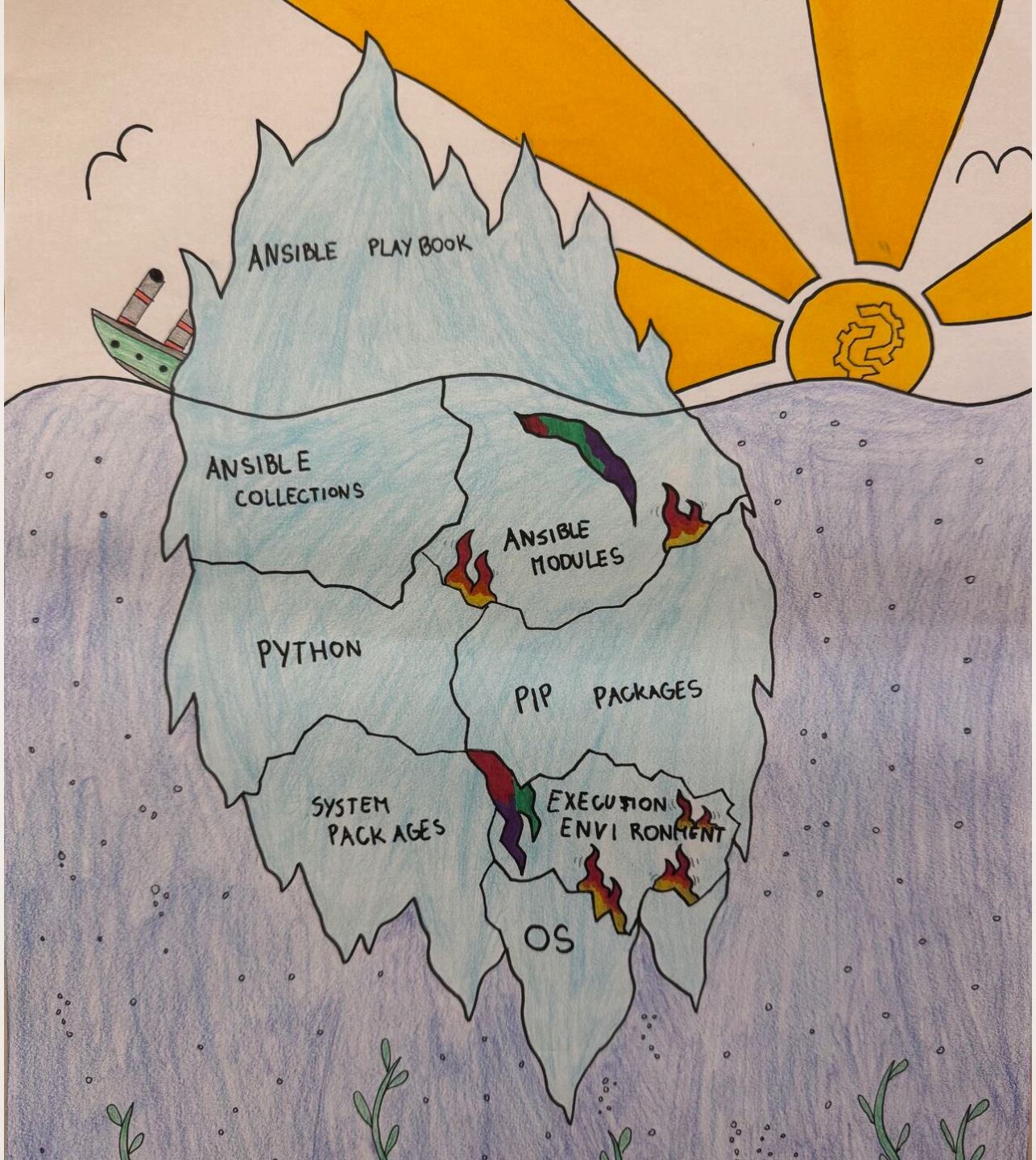
Why this matters?

Playbooks are just one source of failures

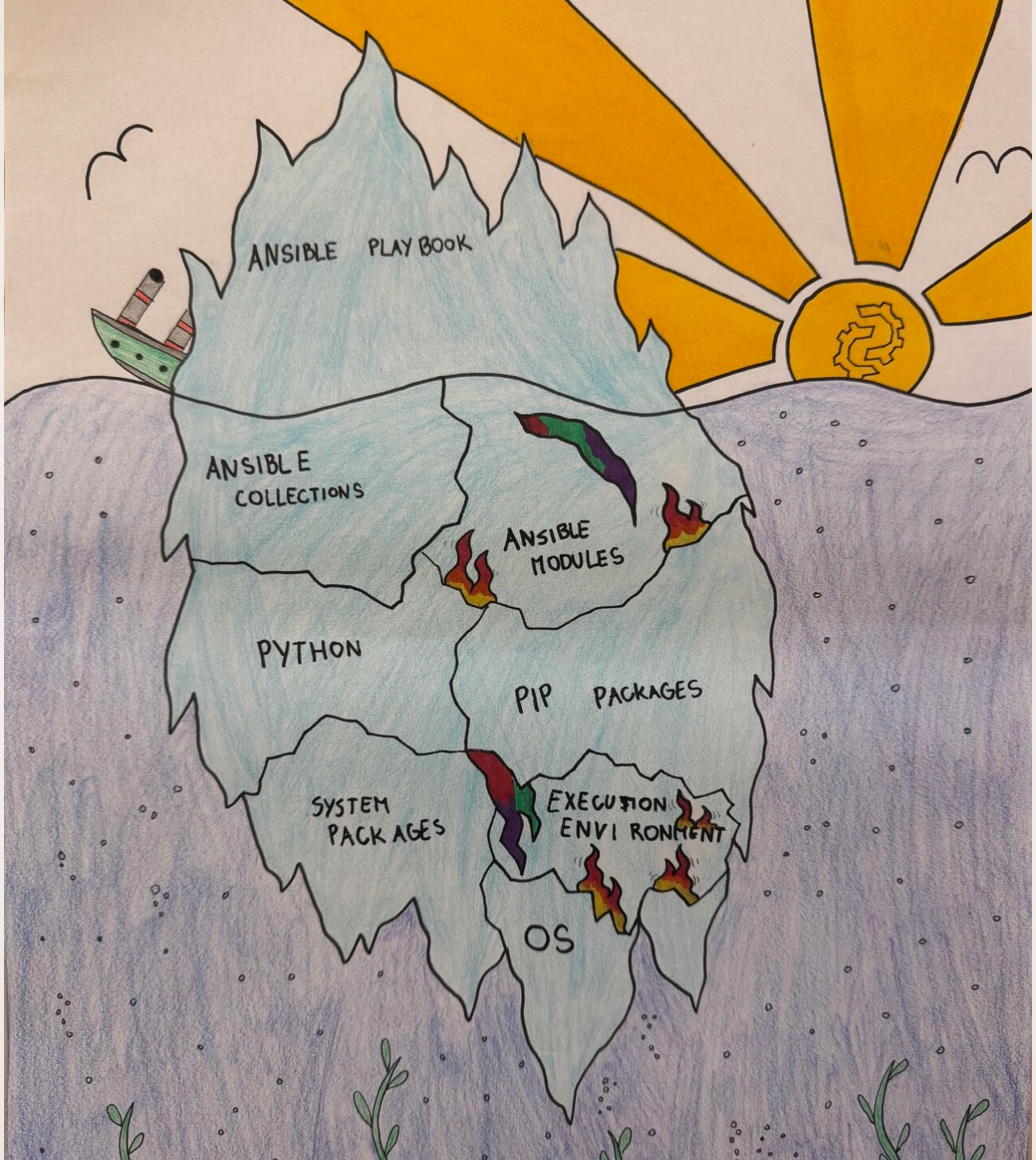
Every layer can introduce risk or instability

Most issues in dependencies are surprises

Developer


ANSIBLE PLAY BOOK
ANSIBLE COLLECTIONS
ANSIBLE MODULES
PYTHON
PIP PACKAGES
SYSTEM PACKAGES
EXECUTION ENVIRONMENT
OS

Developer



Organization

Playbooks are small
Their impact is huge

Typical sources of failures

Sensitive data leaks and resource misconfiguration

Updated module behavior

Changed 3rd party library dependency

Execution environment drift

The Cost of Late Discovery

Issues found in production

Multiple teams pulled in

Rollbacks and exceptions

Meetings and escalations

TIME LOST

Myth

Governance slows teams down

Freedom Early, Discipline Later

Development

- Experiment freely
- Break things
- Learn fast

Production

- Predictability
- Reproducibility
- Shared responsibility

Production is a Contract

Automation in production **affects others**

Platform, security and operations **inherit the risk**

Guardrails are not optional

The Four Pillars of Automation Governance

Visibility

Know what is executed

Include dependencies, not just playbooks

Remove blind spots

Consistency

Same rules for all teams

No special cases

No tribal knowledge

Policy as Code

Human memory does not scale

Expectations must be machine-readable

Policies evolve with the platform

Feedback Loops

Early, automated signals

In developer context

Before production

What this means for Developers?

Full autonomy in development and experimentation

Fast, automated feedback long before hitting the production

Fewer late-stage redesigns and rewrites under pressure

What this means for CI/CD & Platform teams?

Clear, automated guardrails replace ad-hoc exceptions

Fully predictable promotion of automation to production

Less firefighting caused by invisible dependency changes

What this means for Security teams?

Security expectations become machine-enforced, not document-based

Controls apply consistently across automation, not just “critical” ones

Reduced reliability on manual reviews and early visibility into risks before production exposure

Ansible Automation is a Software Supply Chain

At scale it **does not run on trust**

It runs on **governance**

Key Takeaways

Non-secure Ansible Playbooks can quietly introduce **risks that scale across your entire infrastructure**

Security risks don't stop at Playbooks — **vulnerabilities anywhere in the dependency chain can impact every execution**

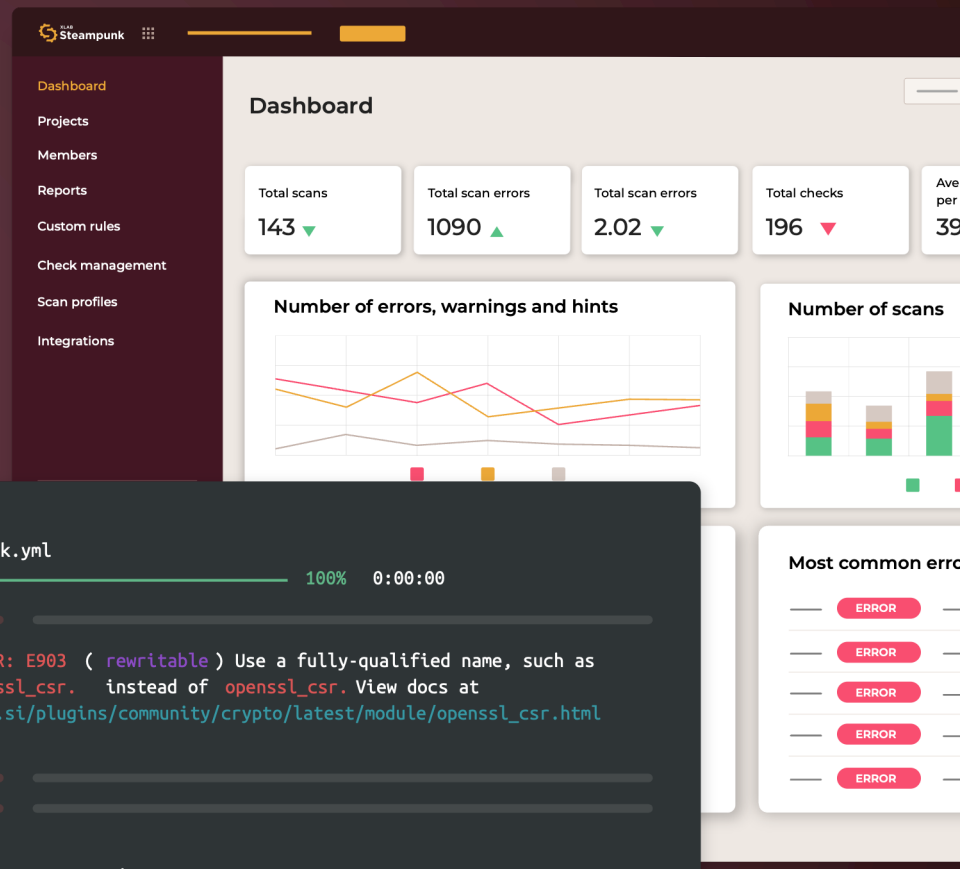
Following Ansible security benchmarks **turns Playbooks into predictable, audit-ready automation**

Automated approach

Steampunk Spotter

Advanced Ansible Playbook Platform

- Observes **automation supply chains**
- Analyzes **risks across layers**
- Enforces **guardrails automatically**




```
> spotter scan playbook.yml
Scanning...success                    100%   0:00:00

playbook.yml:8:7: ERROR: E903 ( rewritable ) Use a fully-qualified name, such as
community.crypto.open ssl_csr.   instead of  openssl_csr. View docs at
https://docs.steampunk.si/plugins/community/crypto/latest/module/openssl_csr.html
for more info.

Scan summary:
Spotter took  1.043 s to scan your input.
It resulted in  9 error(s) , 1 warning(s) and  4 hint(s) .
Can  rewrite 1 file(s)  eith  7 change(s) .
Overall status:  ERROR
```

XLAB Steampunk

XLAB Steampunk

cheers

steampunk.si/spotter/